

17/03/26

Exp. No: 6a

First-Come, First Served CPU scheduling Algorithm

Aim:

To write a C program that implements the First-Come, First Served (FCFS) CPU scheduling algorithm and calculates Completion Time, Turnaround Time, and Waiting Time for each process, along with their averages.

Sample Input:

3

0 2

1 2

2 3

Sample Output:

Enter number of processes:

Enter Arrival Time and Burst Time for P₁:

Enter Arrival Time and Burst Time for P₂:

Enter Arrival Time and Burst Time for P₃:

Average Turnaround Time : 3.33

Average Waiting Time : 1.00

Result:

The program successfully schedules process using the FCFS algorithm and CT, TAT, WT, Average TAT and Average WT based on the given AT and BT inputs.

17/03/26

Exp. No: 6b

Shortest Job First CPU Scheduling Algorithm

Aim:

To implement Non-Preemptive SJF scheduling in C and compute CT, TAT, and WT.

Sample Input:

4

0 6

0 8

0 1

0 3

Sample Output:

Enter number of processes:

Enter Arrival Time and Burst Time for P₁:

Enter Arrival Time and Burst Time for P₂:

Enter Arrival Time and Burst Time for P₃:

Average Turnaround Time : 2.33

Average Waiting Time = 0.33

Result:

Processes are scheduled by shortest burst time, and CT, TAT, WT, and their averages are calculated successfully.

18/03/26

Exp. No: 6c Non-Preemptive Priority Scheduling algorithm

Aim:

To implement Non-Preemptive Priority Scheduling in C and calculate average waiting and turnaround time.

Sample input:

3

0 5 2

1 3 1

2 2 3

Sample Output:

Enter number of processes:

Enter AT, BT and Priority for P₁:

Enter AT, BT and Priority for P₂:

Enter AT, BT and Priority for P₃:

Average Turnaround Time = 6.67

Average Waiting Time = 3.33

Result:

Processes are scheduled by priority, and average waiting time and turnaround time are obtained.

18/03/26

Exp. No: 6d Round Robin CPU Scheduling Algorithm

Aim:

To implement the Round Robin CPU scheduling algorithm in C and calculate CT, TAT, and WT.

Sample Input:

3

2

0 5

1 3

2 1

Sample Output:

Enter number of processes:

Enter Time Quantum:

Enter AT and BT for P₁:

Enter AT and BT for P₂:

Enter AT and BT for P₃:

Average Turnaround Time : 6.33

Average Waiting Time : 3.33

[Signature]

Result:

Processes are executed using time quantum, and CT, TAT, WT along with their averages are calculated.

FACE Prep

Operating System - Coding | REC | 3 Feb

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Time (AT) and Burst Time (BT) as you given from the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT) and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selects out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulas

- Elapsed Time (ET) = Completion Time (CT) - Arrival Time (AT)
- Blocking Time (BT) = Turnaround Time (TAT) - Burst Time (BT)

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The next four sample lines integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

4
0 8
0 8
0 7
0 9

Sample Output 1

Enter number of processes:
Enter Arrival Time and Burst Time for P0:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Average Turnaround Time = 33.50
Average Waiting Time = 7.50

```
100 printf("Enter Arrival Time and Burst Time for P0: ", a[0], b[0]);
101 scanf("%d %d", &a[0], &b[0]);
102 while(completed < n)
103 {
104     int shortest = 9999;
105     int index = -1;
106     for(i = 0; i < n; i++)
107     {
108         if(a[i] <= time && b[i] <= shortest)
109         {
110             shortest = b[i];
111             index = i;
112         }
113     }
114     if(index == -1)
115     {
116         break;
117     }
118     a[index] = a[index] + b[index];
119     b[index] = b[index] - b[index];
120     printf("Enter number of processes: ");
121     scanf("%d", &n);
122     completed++;
123 }
124 }
125 printf("Average Turnaround Time = %.2f", avgTAT);
126 printf("Average Waiting Time = %.2f", avgWT);
127 return 0;
```

Priority:

FACE Prep

Operating System - Coding | REC | 3 Feb

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT) and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.
The next four sample lines integers for each process, where
• The first integer denotes the Arrival Time (AT)
• The second integer denotes the Burst Time (BT)
• The third integer denotes the Priority of the process.
A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

4
0 8 2
0 8 3
0 7 1
0 9 4

Sample Output 1

Enter number of processes:
Enter AT, BT and Priority for P0:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Average Turnaround Time = 8.67
Average Waiting Time = 3.33

Example 2

Sample Input 2

4
0 8 2
0 8 3
0 7 1
0 9 4

```
100 while(completed < n)
101 {
102     int shortest = 9999;
103     int index = -1;
104     for(i = 0; i < n; i++)
105     {
106         if(a[i] <= time && b[i] <= shortest)
107         {
108             shortest = b[i];
109             index = i;
110         }
111     }
112     if(index == -1)
113     {
114         break;
115     }
116     a[index] = a[index] + b[index];
117     b[index] = b[index] - b[index];
118     printf("Enter number of processes: ");
119     scanf("%d", &n);
120     completed++;
121 }
122 }
123 printf("Average Turnaround Time = %.2f", avgTAT);
124 printf("Average Waiting Time = %.2f", avgWT);
125 return 0;
```

FACE Prep

Operating System - Coding | REC | 3 Feb

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT) and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.
The next four sample lines integers for each process, where
• The first integer denotes the Arrival Time (AT)
• The second integer denotes the Burst Time (BT)
• The third integer denotes the Priority of the process.
A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

4
0 8 2
0 8 3
0 7 1
0 9 4

Sample Output 1

Enter number of processes:
Enter AT, BT and Priority for P0:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Average Turnaround Time = 8.67
Average Waiting Time = 3.33

Example 2

Sample Input 2

4
0 8 2
0 8 3
0 7 1
0 9 4

```
100 while(completed < n)
101 {
102     int shortest = 9999;
103     int index = -1;
104     for(i = 0; i < n; i++)
105     {
106         if(a[i] <= time && b[i] <= shortest)
107         {
108             shortest = b[i];
109             index = i;
110         }
111     }
112     if(index == -1)
113     {
114         break;
115     }
116     a[index] = a[index] + b[index];
117     b[index] = b[index] - b[index];
118     printf("Enter number of processes: ");
119     scanf("%d", &n);
120     completed++;
121 }
122 }
123 printf("Average Turnaround Time = %.2f", avgTAT);
124 printf("Average Waiting Time = %.2f", avgWT);
125 return 0;
```


Round Robin:

FACE Prep

Operating System - Coding | REC | 3 Feb

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next three lines contain two integers for each process, where:
• The first integer denotes the Arrival Time (AT).
• The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

```
3
2
0 8
1 4
2 4
```

Sample Output 1

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P0:
Enter AT and BT for P1:
Enter AT and BT for P2:

Average Turnaround Time: 5.33
Average Waiting Time: 3.33
```

Select Language: C (GCC 9.4.0)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n, tq, i, size = 0, completed = 0;
5     int AT[10], BT[10], CT[10], TT[10], WT[10], AT[10];
6     float avgAT = 0, avgBT = 0;
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9     printf("Enter Time Quantum: ");
10    scanf("%d", &tq);
11    for(i = 0; i < n; i++)
12    {
13        printf("Enter AT and BT for P%d: ", i+1);
14        scanf("%d %d", &AT[i], &BT[i]);
15        CT[i] = BT[i];
16    }
17    while(completed < n)
18    {
19        int completed = 0;
20        for(i = 0; i < n; i++)
21        {
22            if(CT[i] < size && BT[i] > 0)
23            {
24                completed++;
25                CT[i] += tq;
26                if(CT[i] < size)
27                {
28                    size = CT[i];
29                    CT[i] = size;
30                    BT[i] -= tq;
31                }
32                else
33                {
34                    size = BT[i];
35                    CT[i] = size;
36                    BT[i] = 0;
37                    completed++;
38                    CT[i] = CT[i] - BT[i];
39                    avgAT += AT[i];
40                    avgBT += BT[i];
41                }
42            }
43        }
44        if(completed == n)
45            break;
46    }
47    printf("Average Turnaround Time: %.2f", avgAT);
48    printf("Average Waiting Time: %.2f", avgBT);
49    return 0;
50 }
```

Test Cases

Custom Test Case

Sample Test Case

Test Case - 1

Test Case - 2

Run Sample Cases

Check Answer

FACE Prep

Operating System - Coding | REC | 3 Feb

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TT), and Waiting Time (WT) for each process and the overall averages.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the Time Quantum.
3. The next three lines contain two integers for each process, where:
• The first integer denotes the Arrival Time (AT).
• The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

```
3
2
0 8
1 4
2 4
```

Sample Output 1

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P0:
Enter AT and BT for P1:
Enter AT and BT for P2:

Average Turnaround Time: 5.33
Average Waiting Time: 3.33
```

Select Language: C (GCC 9.4.0)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n, tq, i, size = 0, completed = 0;
5     int AT[10], BT[10], CT[10], TT[10], WT[10], AT[10];
6     float avgAT = 0, avgBT = 0;
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9     printf("Enter Time Quantum: ");
10    scanf("%d", &tq);
11    for(i = 0; i < n; i++)
12    {
13        printf("Enter AT and BT for P%d: ", i+1);
14        scanf("%d %d", &AT[i], &BT[i]);
15        CT[i] = BT[i];
16    }
17    while(completed < n)
18    {
19        int completed = 0;
20        for(i = 0; i < n; i++)
21        {
22            if(CT[i] < size && BT[i] > 0)
23            {
24                completed++;
25                CT[i] += tq;
26                if(CT[i] < size)
27                {
28                    size = CT[i];
29                    CT[i] = size;
30                    BT[i] -= tq;
31                }
32                else
33                {
34                    size = BT[i];
35                    CT[i] = size;
36                    BT[i] = 0;
37                    completed++;
38                    CT[i] = CT[i] - BT[i];
39                    avgAT += AT[i];
40                    avgBT += BT[i];
41                }
42            }
43        }
44        if(completed == n)
45            break;
46    }
47    printf("Average Turnaround Time: %.2f", avgAT);
48    printf("Average Waiting Time: %.2f", avgBT);
49    return 0;
50 }
```

Test Cases

Custom Test Case

Sample Test Case

Test Case - 1

Test Case - 2

Run Sample Cases

Check Answer